# ChainGuard: A Multi-Agent Smart Contract Security Auditor

Abel S Mathew, Abraham Santhosh, Diya Pauly, Megha Ajith
Alka Vijay
Department of Computer Engineering
College of Engineering Chengannur
APJ Abdul Kalam Technological University
Kerala, India

***Abstract*—Smart contracts control digital assets, and a single undetected flaw in their code can cause irreversible financial loss. Existing audit practice is either expensive and manual, or relies on static analyzers such as Slither that are fast but report many false positives and miss context-dependent logic flaws, while single-pass large language model (LLM) auditors hallucinate and give inconsistent answers. This paper presents ChainGuard, a terminal-based multi-agent smart contract security auditor that connects developers, a static analyzer and several LLM providers in one automated pipeline. Slither findings are filtered and sliced into small line-numbered code contexts by deterministic code. Each finding is then argued by a Prosecutor agent, challenged by a Defender agent and decided by a Judge agent that returns a schema-validated verdict (CONFIRMED, FALSE_POSITIVE or INCONCLUSIVE) with a confidence score and a patch recommendation. A quota-aware router spreads calls across NVIDIA NIM, Groq and Google Gemini free tiers using per-model budget tracking and live model probing, a Pydantic validator with bounded retries guarantees structured output, and a SHA-256 keyed SQLite cache removes repeated work. The system flags verdicts produced in degraded mode and returns INCONCLUSIVE instead of guessing. This paper describes the motivation, related work, system design, functional modules, verification methodology and implementation details of the prototype, states which modules are implemented and which are still in progress, and outlines directions for future enhancement.**

***Index Terms*—Smart contracts, vulnerability detection, Solidity, Slither, large language models, multi-agent systems, multi-agent debate, static analysis, command-line tool, free-tier APIs.**

## I. Introduction

Smart contracts are self-executing programs on blockchain platforms. Once deployed they are hard to modify and often hold substantial value, so vulnerabilities such as reentrancy, improper access control, transaction-order dependence and unsafe use of `tx.origin` have serious consequences [1]–[4]. Professional audits are costly and slow, so many developers, students and small teams deploy contracts with little or no review.

Existing automated tools address only part of the problem. Static analysis frameworks such as Slither [1] are fast and traceable but depend on predefined rules and produce false positives. Learned detectors based on attention models, transformer encoders and ensembles [2]–[4] need labelled data and explain little. LLM-based auditors [5]–[7], [11] read code and reason in natural language, but hallucinate and are sensitive to prompts. Multi-agent debate [8], [9], role-based agent teams [10] and the multi-agent auditor LLM-SmartAudit [12] suggest that agents which challenge one another give more reliable results than one model answering alone.

This project was undertaken as a final-year academic project. Our literature survey concluded that no single technique solves smart contract auditing and that the most promising direction is a hybrid, evidence-driven design in which static analysis proposes candidates, LLM agents reason about them, and independent agents act as prosecutor, defender and judge. ChainGuard implements that direction as a working command-line tool. Two practical constraints shaped it: it must run on a developer's own machine, and it must work with free-tier LLM APIs whose request-per-minute (RPM) and daily limits are strict and change often.

The key contributions of this work are:

- A centralized command-line platform that runs Slither, deterministic filtering and slicing, and a three-agent courtroom in one command.
- A verdict scheme (CONFIRMED / FALSE_POSITIVE / INCONCLUSIVE with confidence and patch) that drives how each finding is reported and cached.
- A deterministic pre-processing layer so that agents receive small, grounded, line-numbered code contexts.
- A quota-aware routing layer with per-model budgets, live model probing, role-based model sizing and degraded-mode flagging.
- Schema-validated output with bounded retries and a content-hash cache, so that malformed output never crashes a run and repeated work costs nothing.

The remainder of this paper is organized as follows. Section II reviews related work. Section III discusses the existing system and its limitations. Section IV presents the proposed system. Section V describes the system architecture. Section VI details the functional modules. Section VII presents the verification methodology. Section VIII discusses implementation details. Section IX discusses expected outcomes, Section X outlines limitations and future scope, and Section XI concludes the paper.

## II. Literature Survey

A wide body of recent work addresses smart contract vulnerability detection through static analysis, machine learning, LLMs and multi-agent systems. Table I summarizes the twelve studies that informed the design of ChainGuard.

Feist et al. [1] introduced Slither, a static analysis framework that uses intermediate representations and a library of security detectors; it is fast and fits development workflows but depends on predefined rules, so semantic logic flaws and unseen patterns remain hard to detect. Wu et al. [2] proposed a hybrid attention

Table I
SUMMARY OF REVIEWED RELATED WORK

| Study | Core Technique | Relevant Limitation |
|---|---|---|
| [1] | Slither static detectors, IR | False positives; weak on logic flaws |
| [2] | Hybrid attention classifier | Depends on training data; little explanation |
| [3] | Fine-tuned transformer encoder | Needs labelled data; limited reasoning |
| [4] | Ensemble multi-label learning | Distribution shift; labelled data needed |
| [5] | LLM-based detection (GPTLENS) | Hallucination; prompt sensitivity |
| [6] | Custom generative AI (SmartLLM) | Evaluation tied to chosen dataset |
| [7] | GPT plus program analysis | Bounded by analysis coverage |
| [11] | Prompt engineering (CoT, few-shot) | Example sensitivity; small context |
| [8], [9] | Multi-agent debate, ChatEval | Extra cost; no correctness guarantee |
| [10] | Role-based agents (ChatDev) | Not security-specific |
| [12] | LLM-SmartAudit multi-agent | Coordination cost; evidence validation |

mechanism model that learns code representations automatically, and Le et al. [3] fine-tuned transformer encoders for Solidity; both depend on training data and explain little. Alnuman et al. [4] used an ensemble multi-label model, important because one contract can contain several vulnerability types, and reported F1 scores from about 82.0% to 99.9%, though supervised models can struggle on patterns outside the training distribution. Hu et al. [5] explored LLM-powered detection (GPTLENS), showing the value of broader context and natural-language explanations along with hallucination and prompt sensitivity. Kevin and Yugopuspito [6] proposed SmartLLM, which reported 70.0% accuracy and a 76.9% F1 score against 21.7% for Mythril and 35% for Slither in their own evaluation. Sun et al. [7] combined GPT with program analysis (GPTScan) to find logic vulnerabilities, using analysis to select the code regions that the LLM reasons about. Su et al. [11] showed that prompt design, including Chain-of-Thought and few-shot prompting, substantially changes detection performance.

On the multi-agent side, Du et al. [8] showed that language-model agents that debate improve factuality and reasoning. Chan et al. [9] introduced ChatEval, in which role-diverse evaluator agents give more reliable judgements, and Qian et al. [10] introduced ChatDev, which organizes role-based agents for software development. Wei et al. [12] applied multi-agent collaboration directly to smart contract auditing (LLM-SmartAudit), at the cost of extra inference, coordination complexity and a need to verify each agent's evidence.

*Research Gap:* Across the reviewed literature, individual studies rely on rules alone, on supervised classifiers alone, on a single LLM pass, or on large multi-agent teams, but none combines (i) a static analyzer as candidate generator, (ii) deterministic code slicing so that agents see only grounded evidence, (iii) a fixed adversarial prosecutor–defender–judge debate with typed, validated outputs, and (iv) a quota-aware routing layer that makes the whole pipeline usable on a developer's machine with free-tier APIs. ChainGuard is designed to address this gap.

## III. EXISTING SYSTEM AND ITS LIMITATIONS

In the currently prevalent audit practice:

- Manual expert audits are expensive and slow, and are out of reach for many small projects.
- Static analyzers report every pattern match, so developers must triage many false positives by hand.
- Static rules cannot judge application-specific logic or guard conditions.
- Single-pass LLM audits can state unsupported findings with confidence and may answer differently on each run.
- Raw LLM output is unstructured, so malformed answers break automated pipelines.
- Free-tier API limits cause failures mid-run, with no fallback or record of which results are trustworthy.
- Repeated audits of the same contract repeat the same cost.

These limitations lead to alert fatigue, missed logic vulnerabilities, unreliable explanations and wasted API budget.

## IV. PROPOSED SYSTEM

The proposed tool automates the audit of a Solidity file using deterministic analysis, intelligent multi-agent reasoning and resilient resource management.

### A. Courtroom-Based Verification

Unlike single-pass LLM auditors [5], [11], ChainGuard treats every Slither finding as a case. A Prosecutor argues that the finding is a real, exploitable vulnerability, a Defender looks for guards or constraints that block the exploit, and a Judge weighs both against the actual code. This follows the role-based collaboration of [10], [12] and the debate principle of [8], [9], but with a fixed three-role structure and typed hand-offs.

### B. Verdict Classification

Each case ends in one of three verdicts. CONFIRMED means the vulnerability is judged real and a patch recommendation is produced. FALSE_POSITIVE means the code context shows the finding is not exploitable. INCONCLUSIVE means evidence is insufficient or the pipeline failed, and the finding is left for manual review. The Judge also returns a confidence score from 0.0 to 1.0 (certainty in the verdict, not severity) and an optional final severity.

### C. Core System Capabilities

The tool runs Slither automatically with the compiler version taken from the contract's `pragma`; filters Informational and Optimization noise and ranks the remaining findings; slices only the relevant functions with line numbers; routes each agent call to a provider with spare quota; validates every response against a schema and retries with correction context; caches genuine verdicts so unchanged contracts cost nothing; flags verdicts produced in degraded mode; and writes the full debate record of every finding as JSON.

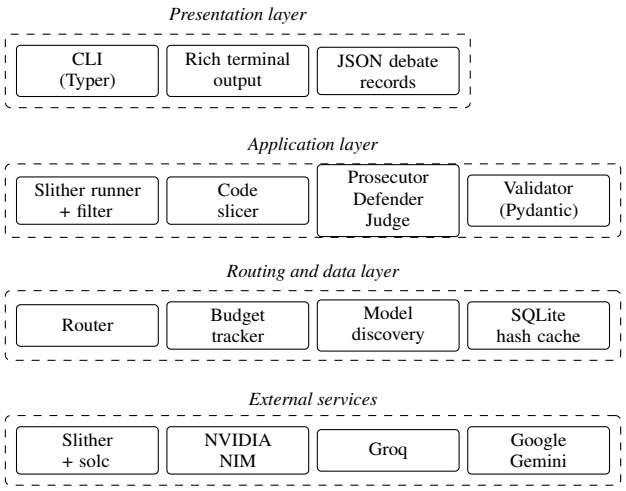


Figure 1. Layered system architecture of ChainGuard.

## V. System Architecture

The system follows a layered architecture with four layers, illustrated in Fig. 1: (i) a presentation layer, the `smart-audit` command-line interface with Rich terminal output and JSON export; (ii) an application layer hosting the Slither runner and filter, the code slicer, the courtroom agents and the validator; (iii) a routing and data layer with the router, budget tracker, model discovery and the SQLite cache; and (iv) an external services layer comprising Slither with solc and the NVIDIA NIM, Groq and Google Gemini APIs. Stages up to slicing involve no LLM calls.

## VI. Functional Modules

The tool is organized into the following functional modules:

1. *Command-Line Interface* – `smart-audit run <file>.sol`, path resolution, `.sol` validation and API keys from `.env`.
2. *Slither Runner* – runs Slither in JSON mode and installs the solc version required by the contract's `pragma`.
3. *JSON Filter* – drops Informational and Optimization results, ranks the rest by impact then confidence and applies an optional `-max-findings` cap.
4. *Code Slicer* – extracts line-numbered function code around each reported span, padded by three lines, with overlapping spans merged.
5. *Prosecutor Agent* – produces a charge with reasoning, severity assessment and attack scenario.
6. *Defender Agent* – produces a good-faith rebuttal or concedes the charge.
7. *Judge Agent* – produces the verdict, confidence, severity and patch recommendation.
8. *Validator* – parses JSON, validates schemas, retries with correction context and falls back to INCONCLUSIVE.
9. *API Router* – selects provider and model by role tier and records degraded mode.
10. *Budget Tracker and Model Discovery* – per-model RPM and daily limits, and live probing of candidate models with a 24 hour cache.
11. *Hash Cache* – stores genuine verdicts keyed by contract source and finding.
12. *Output and Reports* – Rich terminal summary and JSON debate records; a Markdown report generator is planned.

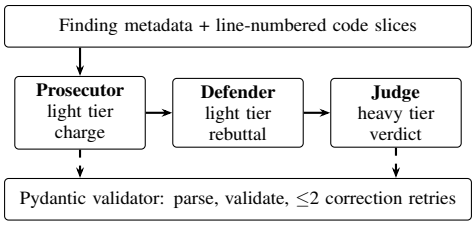


Figure 2. Single-pass courtroom. Each agent output is validated; the Judge falls back to INCONCLUSIVE (confidence 0.0) if validation keeps failing.

## VII. Verification Methodology

### A. Finding Prioritization

Slither findings are ordered by a tier in which Slither's impact is the primary key and its confidence the secondary key, so the Judge's limited daily budget is spent on the most serious cases first. A cap $K$ (`-max-findings`) limits how many are processed.

### B. Courtroom Procedure

For a retained finding $f$ with code context $x_f$, the three agents run once in sequence:

$$c = P(f, x_f), \quad r = D(f, x_f, c), \quad v = J(f, x_f, c, r) \qquad (1)$$

where $P$, $D$ and $J$ are the Prosecutor, Defender and Judge, and $v = (\text{verdict}, \text{severity}, \gamma, \text{patch})$ with confidence $\gamma \in [0,1]$. Fig. 2 shows the flow. There is no re-loop; a Defender that finds the charge sound concedes, and the Judge weighs both arguments against the code rather than deferring to the more confident side.

### C. Quota-Aware Routing

For each (provider, model) pair $m$ the tracker keeps requests used per minute and per day. Let $u(m) = \max\left(\frac{\text{rpm}_m}{\text{RPM}_m^{\max}}, \frac{\text{day}_m}{\text{DAY}_m^{\max}}\right)$ be its utilization. Light roles (Prosecutor, Defender) prefer small fast models with provider order Groq, Gemini, NVIDIA NIM; the heavy role (Judge) prefers larger models with order NVIDIA NIM, Groq, Gemini. The router selects the candidate with the most headroom, $\arg\min_m u(m)$, among those below the soft threshold $u(m) < 0.85$, using provider order to break ties. If it must use a candidate above the threshold or a non-first-choice provider, the record is marked `degraded_mode`. A pair that fails a real call is skipped for the rest of the run but not blacklisted.

### D. Caching

A verdict is stored under $k = \text{SHA-256}(C \,\|\, f.\text{id})$, where $C$ is the contract source. A hit bypasses all LLM calls. Synthesized INCONCLUSIVE records caused by transient failures are not cached, so they are retried on the next run.

Table II
TECHNOLOGY STACK

| Component | Technology |
|---|---|
| Interface | Python 3.10+, Typer, Rich |
| Static analysis | Slither, py-solc-x |
| Validation | Pydantic v2 schemas |
| LLM access | NVIDIA NIM, Groq, Google Gemini (REST via requests) |
| State | SQLite (cache, budgets), JSON model catalog |
| Testing and style | pytest, black, ruff |
| Version control | Git and GitHub |

## VIII. IMPLEMENTATION DETAILS

### A. Hardware Requirements

A general-purpose computer able to run Python 3.10 or later and the Solidity compiler, with an active internet connection for the LLM APIs. No GPU is required because inference runs on the providers' servers.

### B. Software Requirements

Python 3.10+, Slither (slither-analyzer), py-solc-x, Typer, Rich, Pydantic v2, requests, python-dotenv, SQLite, and API keys for NVIDIA NIM, Groq and Google Gemini. Development tooling is pytest, pytest-mock, black, ruff and Git/GitHub for version control.

**Algorithm 1:** Courtroom audit of one contract
**Require:** contract source $C$, path $p$, cap $K$
$raw \leftarrow$ Slither($p$), JSON mode
$F \leftarrow$ Filter($raw$), ranked, first $K$ kept
$S \leftarrow$ Slice($F$, $p$), pad 3 lines, merge spans
**for each** finding $f \in S$:
  **if** cache has SHA-256($C \| f.id$): reuse, **continue**
  $c \leftarrow$ Prosecutor($f$); validate (retry $\leq 2$)
  $r \leftarrow$ Defender($f$, $c$); validate (retry $\leq 2$)
  $v \leftarrow$ Judge($f$, $c$, $r$), else INCONCLUSIVE
  cache genuine $v$ and append the record
**Ensure:** debate record for every retained finding

The courtroom agents are single-purpose modules with typed schemas (`ProsecutorCharge`, `DefenderResponse`, `JudgeVerdict`) combined in a `DebateRecord` that also stores a contract hash and the degraded-mode flag. SQLite runs in write-ahead logging mode and is the single source of truth for provider state, so the router is stateless between calls.

### C. Development Highlights

The repository history shows the routing layer changing as the system met real free-tier behaviour. The first design named NVIDIA NIM, Mistral and OpenRouter; Mistral was removed as unreliable, OpenRouter was dropped, and Groq and Gemini were added. Hard-coded model names proved fragile and were replaced by live discovery with probing. Budgets moved from per-provider to per-model because quotas differ widely between models of one provider, and the single timeout was split into 25 s for light and 40 s for heavy models. The lesson is that the part of an LLM auditing tool most exposed to change is the layer that keeps agents reachable, within budget and well formed.

Table III
MODULE IMPLEMENTATION STATUS

| Module | Status |
|---|---|
| CLI, Slither runner, solc selection | Implemented |
| JSON filter and code slicer | Implemented |
| Prosecutor, Defender, Judge, schemas | Implemented (single pass) |
| Validator, retry, INCONCLUSIVE fallback | Implemented |
| Router, budget tracker, model discovery | Implemented |
| SQLite hash cache | Implemented |
| JSON debate records | Implemented |
| Markdown report generator | Planned (module empty) |
| `-fast` delay buffers | Planned (flag is a no-op) |
| Benchmark evaluation | In progress |

Table IV
RESULT TABLE TO BE COMPLETED AFTER BENCHMARK RUNS

| System | P | R | F1 | FP red. |
|---|---|---|---|---|
| Slither only | TBD | TBD | TBD | n/a |
| Single-agent LLM, slices | TBD | TBD | TBD | TBD |
| ChainGuard (P–D–J) | TBD | TBD | TBD | TBD |

## IX. EXPECTED OUTCOMES AND DISCUSSION

As an academic prototype, ChainGuard is expected to demonstrate: (i) fewer false positives for the developer to triage than raw Slither output, because the Defender and Judge test each finding against the code; (ii) more reliable automation, because schema validation turns malformed model output into an explicit INCONCLUSIVE result instead of a crash; (iii) lower cost on repeated audits, because unchanged contracts hit the cache; and (iv) transparent results, because every verdict traces to a Slither detector, a line-numbered slice and three stored arguments, and degraded-mode verdicts are marked.

No benchmark measurements have been collected yet, and none are claimed. We therefore define the evaluation protocol first. Datasets would be labelled real-world Solidity contracts, for example SmartBugs samples as used in [11], with a held-out set not used for prompt tuning. Baselines are Slither alone and a single-agent LLM that sees the same slice. Ablations remove the Defender, remove slicing and remove the retry and cache layer. Metrics are precision, recall and F1 per class, false-positive reduction against Slither, the INCONCLUSIVE rate, agreement between confidence and correctness, calls and latency per contract, and the fraction of degraded findings. Table IV is to be completed after these runs.

## X. LIMITATIONS AND FUTURE SCOPE

### A. Limitations

The courtroom can remove false positives but cannot find a vulnerability that Slither does not flag, so logic flaws outside any detector remain uncovered. Slices omit inherited contracts, modifiers and cross-contract calls, so the Defender can miss a guard outside the slice. The debate is single-pass with no

independent evidence check, and the Judge can inherit a wrong assumption shared by both lighter agents. Free-tier limits and model names change, so candidate lists need upkeep. Contract code is sent to third-party APIs, which may be unacceptable for confidential code, and the tool audits one `.sol` file per run.

### B. Future Scope

The system can be further enhanced by:

- Implementing the Markdown and JSON report generator and the `-fast` delay buffers.
- Running the evaluation protocol on a labelled benchmark with ablations.
- Requiring code-level evidence for CONFIRMED verdicts and generating exploit tests as an independent check, as proposed in our survey.
- Adding inherited contracts, modifiers and called functions to slices through compiler-aware analysis.
- Adaptive agent selection, parallel execution and early stopping to reduce cost.
- Retrieval of known vulnerability patterns and support for local models so that code stays on the machine.
- Multi-file projects and additional static analyzers as candidate generators.

## XI. Conclusion

This paper presented ChainGuard, a terminal-based smart contract security auditor that combines Slither's deterministic candidate generation with a Prosecutor–Defender–Judge courtroom of LLM agents. Deterministic filtering and slicing keep prompts small and grounded, typed schemas with bounded retries make agent output reliable, and a quota-aware router with model probing, per-model budgets and a verdict cache lets the pipeline run under strict free-tier limits while flagging degraded results. The prototype covers the full path from a Solidity file to schema-validated debate records, while the report generator and quantitative evaluation remain in progress. The project promotes secure, transparent and accessible smart contract development, and can be extended into a more evidence-driven auditing platform.


## Acknowledgment

The authors sincerely thank their project guide, Alka Vijay, for her valuable guidance and support throughout the course of this project, and the researchers whose studies were reviewed.


## References


[1] J. Feist, G. Grieco, and A. Groce, "Slither: A static analysis framework for smart contracts," in *Proc. IEEE/ACM Int. Workshop Emerging Trends Softw. Eng. Blockchain (WETSEB)*, 2019.

[2] H. Wu, H. Dong, Y. He, and Q. Duan, "Smart contract vulnerability detection based on hybrid attention mechanism model," *Appl. Sci.*, 2023.

[3] T.-T.-H. Le *et al.*, "Robust vulnerability detection in Solidity-based Ethereum smart contracts using fine-tuned transformer encoder models," *IEEE Access*, 2024.

[4] R. Alnuman, T. Sajid, W. Almobaideen, and Q. Hasan, "Ensemble multi-label machine learning Solidity smart contract vulnerability detection model," *Cluster Comput.*, vol. 28, 2025.

[5] S. Hu *et al.*, "Large language model-powered smart contract vulnerability detection: New perspectives," arXiv:2310.01152, 2023.

[6] J. Kevin and P. Yugopuspito, "SmartLLM: Smart contract auditing using custom generative AI," arXiv:2502.13167, 2025.

[7] Y. Sun *et al.*, "GPTScan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng. (ICSE)*, 2024.

[8] Y. Du *et al.*, "Improving factuality and reasoning in language models through multiagent debate," arXiv:2305.14325, 2023.

[9] C.-M. Chan *et al.*, "ChatEval: Towards better LLM-based evaluators through multi-agent debate," arXiv:2308.07201, 2023.

[10] C. Qian *et al.*, "ChatDev: Communicative agents for software development," arXiv:2307.07924, 2023.

[11] I.-F. Su, S.-M. Wang, Y.-C. Chung, and Y.-H. Tsai, "A smart contract vulnerability detection manner based on large language model," in *Proc. ICKII 2024*, Lecture Notes Elect. Eng., vol. 1481, pp. 16–25, 2026, doi: 10.1007/978-981-95-2113-5_2.

[12] Z. Wei *et al.*, "Advanced smart contract vulnerability detection via LLM-powered multi-agent systems," *IEEE Trans. Softw. Eng.*, vol. 51, no. 10, pp. 2830–2846, 2025, doi: 10.1109/TSE.2025.3597319.